FULL PROJECT ROADMAP

AI-Assisted Weekly Planning SaaS

A practical implementation plan for the Xaventra Software Engineering Internship Exercise

Assessment reference: SE-INT-04 | Starting point: Launchbase monorepo | Estimated effort in brief: ~12 hours

PRIMARY GOAL Deliver a secure multi-tenant planning product with organizations, teams, weekly plans, two useful AI workflows, a staff-only internal console, a coherent brand, and a public landing page.

How to use this roadmap

- Implement in the sequence shown. Complete and test the security boundaries before building more UI.
- Use the coding agent to inspect, draft, implement, and test, but review its data model, authorization checks, and AI outputs yourself.
- Keep optional features out of scope until every required acceptance criterion passes.
- The assignment estimate of 12 hours is a timebox, not a guarantee. Reduce visual polish or optional work before reducing security and test coverage.

Assessment priorities

Area	Weight in brief	Roadmap implication
Tenancy and access control	★★★	Highest priority. Enforce organization isolation, role permissions, team-lead approval rules, and staff-only access in API endpoints.
AI integration	★★★	Highest priority. Use typed Pydantic outputs, practical findings, graceful failure handling, usage logs, and tests without Google network calls.
Product and domain	★★	Build the weekly plan lifecycle, task statuses, team weekly overview, and history.
Internal admin panel	★★	Provide organization and user management, suspension/deactivation, and AI usage visibility.
Founder thinking and brand	★★	Make the customer, one-line pitch, name, SVG logo, and landing page consistent.
Code quality and judgement	★ each	Follow Launchbase patterns, maintain migrations and generated API client, and explain sensible scope cuts.

1. Project scope and definition of done

The finished product should let people register with email and password, create or join an organization, work in teams, draft a weekly plan, receive AI feedback, submit the plan to a team lead, track tasks, and review previous weeks. Platform staff need a separate internal console.

Required area	Expected outcome
Brand and marketing	A distinctive name, one-sentence pitch, SVG logo, branded title/favicon/header/auth screens, internal-panel label, responsive landing page, metadata, and Open Graph image.
Authentication and organizations	Email/password sign-up, organization creation, copyable invite link, owner/admin/member roles, organization settings, membership management.
Teams	Organization admins create teams, add members, assign a team lead, and support users who belong to multiple teams.
Weekly plans	One plan per member per Monday-start week. It contains one main task, sub-goals, and go-to-market goals, each with its own task status.
Approval and tracking	Draft, submitted, approved, and done lifecycle. Leads approve or return plans with a comment. Members track task completion and record whether the main goal shipped.
Team overview	A selected week shows submissions, approvals, task completion, main-goal shipment, and prior weeks.
AI workflows	Plan review with task-specific findings and suggested rewrites. Brain-dump conversion into an editable, pre-filled plan.
Internal admin	Organization metrics and suspension, user search and deactivation, AI calls/failures/rate limits by organization.
Demo and submission	Seed command, realistic demo data, role-based demo logins, SUBMISSION.md, committed code, and assessment.zip created with git archive.

Explicitly out of scope

- Billing and payments, sending real email, email verification, password-reset email, social login, SSO, GitHub integration, notifications, and deployment.

2. Technical structure and setup

Path	Existing Launchbase foundation	Build in this area
api/	FastAPI, async SQLAlchemy, PostgreSQL, Alembic, JWT authentication, pytest, public and internal API surfaces.	Sign-up, organizations, memberships, teams, plans, authorization, AI agents, AI usage records, internal endpoints, seed command.
app/	Customer-facing TanStack Start app, login, auth-guarded sidebar, typed client generated with orval.	Organization setup, team management, plan editor, review and draft AI flows, approvals, task tracking, weekly overview, history.
internal/	Staff console on the /internal surface with user-management foundation.	Platform staff dashboard, organization and user controls, AI usage view.
site/	Statically generated marketing site with placeholder and shadcn/ui components.	Product landing page with coherent copy, brand identity, plan flow explanation, and rendered AI review example.

Environment and repository rules

- Use Linux or macOS. On Windows, use WSL 2 with Ubuntu 24.04 and keep the repository inside the WSL filesystem, not /mnt/c.
- The brief specifies Python 3.12 with uv, Node.js 24+, PostgreSQL 16, and Docker for the test suite.

- Read the root README.md, each folder README.md, and every AGENTS.md before changing the code. For Claude Code, make CLAUDE.md reference @AGENTS.md.
- Use uv run for Python commands, generate Alembic migrations, follow the existing service/route transaction pattern, and regenerate the typed frontend client with npm run generate. Do not hand-write fetch calls when the generated client is expected.
- Commit in small, meaningful steps. Ask the coding agent to plan the schema and access rules before it edits multiple files.

Suggested execution rhythm

1. Inspect the template and record its conventions before implementation.
2. Ask the coding agent to propose a domain model, API boundaries, and authorization matrix. Review these decisions before coding.
3. Build one vertical slice at a time: database, API, generated client, UI, tests.
4. After each slice, run the relevant tests and manually try forbidden access with another organization's IDs.
5. At the end, run the full test suite, use a clean checkout or fresh extracted archive to verify setup, and complete the submission notes.

3. Proposed domain model and access rules

The following is an implementation proposal derived from the required product behaviour. Match names and relationships to Launchbase conventions before creating migrations.

Entity	Key fields / relationships	Important constraints
User	id, email, password hash, is_admin, is_active	is_admin means platform staff only. Staff accounts are not created by public sign-up.
Organization	id, name, status, created_at, last_active_at	Suspended organizations lock out members until reinstated.
OrganizationMembership	organization_id, user_id, role	Unique organization/user pair. Role is owner, admin, or member. Every query must be scoped to authorized organizations.
Invitation	organization_id, token or token hash, created_by, status	Invite joins an organization. Validate the token and organization on the API. Email delivery is not required.
Team	id, organization_id, name, lead membership/user reference	Team must belong to an organization. Lead must be a valid member of that organization and team.
TeamMembership	team_id, user_id	Support multiple teams per user. Prevent duplicate memberships and cross-organization links.
WeeklyPlan	organization_id, member_id, week_start, status, review comment, shipped outcome	week_start is the Monday date. Enforce a single plan per member per week within the organization.
PlanTask	plan_id, type, title, details/why, status, order	Types: main task, sub-goal, go-to-market goal. Each task has its own pending/done state. A plan has one main task.
AIUsageLog	organization_id, user_id, feature, result status, failure type, created_at	Record success, failure, and rate limiting. Avoid storing plan text in staff dashboards unless there is a justified need.

Authorization matrix

Actor	Permitted actions	Must be blocked
Organization owner	Manage own organization settings and membership; issue invitations.	Any organization or internal resource outside their own organization.
Organization admin	Manage own organization members/settings and create/manage its teams.	Platform admin controls, other organizations, and approval actions unless they are also the assigned team lead.
Member	Create and edit own drafts, request AI help, submit plans, tick off own tasks, report whether the main goal shipped.	Other members' private plan changes, team administration, approving plans without lead authority.

Actor	Permitted actions	Must be blocked
Team lead	See the assigned team's weekly status, approve or return its members' submitted plans with feedback.	Approving plans for teams they do not lead or acting outside their organization.
Platform staff	Use /internal to search users, inspect aggregate organization metrics, suspend/reactivate organizations, deactivate accounts, inspect AI usage.	Public customer role elevation. Keep this surface separate from organization admin permissions.

SECURITY RULE Tenant isolation is a backend responsibility. Resolve the authenticated user, organization membership, role, team relationship, and resource ownership on every protected request. Never rely on hidden buttons or IDs supplied by the browser.

Plan state transitions

Current state	Action	Next state / rule
Draft	Member submits	Submitted. Validate that required plan content exists.
Submitted	Assigned team lead approves	Approved. Save reviewer and timestamp if supported by the schema.
Submitted	Assigned team lead returns with comment	Return to Draft for editing, retaining the feedback. Use a dedicated returned state only if you deliberately document the deviation.
Approved	Member completes the week	Done. Record whether the main task shipped and preserve task completion statuses.

4. Implementation roadmap and milestones

Use the phases below as the build sequence. Timeboxes are an aggressive interpretation of the brief's ~12-hour estimate. The work may take longer, especially while debugging the starter repository or learning the agent framework.

Phase	Target effort	Primary result	Priority
0. Discovery and setup	0.75 h	Runnable baseline, conventions understood, coding agent configured.	Required
1. Product identity	0.5 h	Name, pitch, SVG logo, brand decision.	Required
2. Schema and authentication	1.5 h	Migrations, sign-up, organization creation, role/membership foundations.	Critical
3. Teams and planning	2.0 h	Teams, week plans, tasks, review workflow, team overview endpoints.	Critical
4. Security and tests	1.25 h	Tenant boundary tests and permission tests integrated into each slice.	Critical
5. AI integration	1.75 h	Typed plan reviewer and brain-dump-to-plan agent with logging and offline tests.	Critical
6. Internal panel	1.0 h	Organization controls, user search/deactivation, AI usage metrics.	High
7. Customer UI and landing page	1.25 h	Working core screens and responsive branded marketing site.	High

Phase	Target effort	Primary result	Priority
8. Seed, acceptance QA, submission	2.0 h	Demo data, full checks, SUBMISSION.md, clean assessment archive.	Required

Phase 0 - Inspect Launchbase and prepare the repository

Objective: Establish a reliable baseline before feature work.

- Clone the repository into the local folder named for your product. Run the documented setup and record the known-good start commands.
- Read the root and folder README files and AGENTS.md instructions. Map the authentication flow, current database models, migrations, service pattern, API routes, generated client workflow, and /internal guard.
- Run existing tests and frontend checks before edits. Record existing failures separately from new failures.
- Configure the coding agent to read the repository instructions and to propose plans before broad changes.

Exit criteria: The app starts locally, existing conventions are understood, and baseline test results are recorded.

Phase 1 - Choose a product identity

Objective: Make the audience and promise clear before building screens.

- Define the target customer, such as engineering team leads and small product teams that want weekly outcomes and clear accountability.
- Write one sentence describing who the product serves and what it helps them accomplish. Use this as the landing-page hero foundation.
- Shortlist names, check for an obvious existing product in the same space, choose one, and record why it fits.
- Create an SVG logo that remains legible at 16 px, 32 px, and large sizes on light and dark backgrounds.

Exit criteria: Name, pitch, rationale, and logo are approved and ready to use consistently across site, app, auth screens, favicon, and internal panel.

Phase 2 - Build the database foundation and sign-up

Objective: Support independent organizations without mixing platform staff with customer roles.

- Create or update SQLAlchemy models and autogenerated Alembic migrations for organizations, memberships, invitations, teams, plans, tasks, and AI usage as appropriate.
- Implement email/password sign-up and organization creation. Add a secure way for an owner to copy an invitation link.
- Enforce unique constraints for membership pairs and one weekly plan per member/week. Validate organization relationships across teams and tasks.
- Implement role checks in backend dependencies or shared authorization services. Keep global platform staff status separate from an organization membership role.
- Add tests for unauthenticated access, inactive users, invalid invite tokens, and cross-organization IDs.

Exit criteria: A user can sign up and create or join an organization. Migrations apply cleanly, and basic tenant/role checks are tested.

5. Core feature delivery

Phase 3 - Teams, weekly plans, and lifecycle

Objective: Deliver the actual weekly planning workflow before adding polish.

- Allow organization admins to create teams, add team members, and assign a lead. Users can belong to more than one team.
- Represent each plan as structured records. Include one main task, multiple sub-goals, and multiple go-to-market goals. Each task has its own completion status.
- Key a plan to the Monday date of the relevant week and prevent duplicate plans for the same member and week.

- Implement draft, submitted, approved, and done states. A team lead approves or returns a submitted plan with a comment. A member can track tasks and record whether the main task shipped.
- Provide team-week summary data for submissions, approvals, tasks complete, shipment status, and previous weeks.
- Regenerate the orval client after API changes and use the generated typed client in the app.

Exit criteria: A member can create, submit, revise, and complete a weekly plan. A lead can review only eligible team plans and see a weekly overview.

Phase 4 - Prove tenancy and authorization

Objective: Close access-control gaps before expanding the feature surface.

- Write tests that create at least two organizations and attempt cross-organization reads and writes using known resource IDs.
- Test that a regular member cannot manage settings or members, a customer admin cannot use /internal, and a non-lead cannot approve a plan.
- Test team membership and plan ownership, including a lead who belongs to multiple teams.
- Test suspended organizations and deactivated users. Deny normal customer access while those controls are active.
- Review list, detail, update, delete, export-like, and AI endpoints. Apply the same scoping logic to every route.

Exit criteria: Automated negative tests prove forbidden operations fail at the API layer, even if a caller knows another user's IDs.

Phase 5 - Implement both AI workflows

Objective: Make the model produce structured results that the product uses directly.

- Configure Pydantic AI with Google Gemini Flash-Lite, using the model ID named in the brief: gemini-3.5-flash-lite. Keep GOOGLE_API_KEY in api/.env and add an empty value to api/.env.example.
- Design Pydantic output models. Plan review should contain a verdict and findings with the affected task, problem, and concrete suggested rewrite.
- Review for tasks phrased as work rather than verifiable results, a missing reason for the main task, missing go-to-market goals, and scope too large for one week.
- Render findings beside the relevant tasks. Let the member apply a suggestion explicitly with one click.
- Implement brain-dump conversion to structured plan data. Fill the editable form only. Never write the model's output directly to the database.
- Log each AI call against the organization with feature, success/failure, and rate-limit outcome. Do not send a model request on every keystroke.
- Handle slow requests, provider outages, malformed output, and HTTP 429 with a clear user message. Manual planning must continue to work.
- Use Pydantic AI TestModel or FunctionModel to test agent logic offline without a key or network access.

Exit criteria: Both flows return validated typed data, help with meaningful planning decisions, degrade gracefully, log usage, and pass offline tests.

Recommended AI output shapes

Workflow	Output fields to consider	UI behaviour
Plan review	verdict; findings[] with task reference or stable task identifier, problem category, explanation, suggested_rewrite	Show each finding by its task. Let the member apply the proposed rewrite or ignore it. AI output does not approve the plan.
Brain dump	main_task; why_it_matters; sub_goals[]; go_to_market_goals[]; optional uncertainties/questions	Map validated output into a draft form. Member reviews and edits before saving or submitting.

AI FALLBACK If the model is unavailable or rate-limited, show a plain message and preserve the user's input. The core planning functions should remain usable without AI.

6. Internal console, product interface, and landing page

Phase 6 - Build the staff-only internal admin panel

Objective: Give platform staff the controls needed to operate the SaaS.

- Use the existing /internal API surface and internal token scope. Platform staff are created through the documented admin-user command, not through public sign-up.
- Show every organization with member count, team count, plan count, and last active time.
- Allow staff to suspend and reinstate organizations. A suspended organization must block its members from using the product.
- Add user search, organization membership visibility, and account deactivation.
- Show AI calls, failures, and rate-limited requests by organization. Make a deliberate data-minimization decision about whether staff need to see plan text at all.
- Use clear names for platform staff authority versus organization admin role.

Exit criteria: Only platform staff can access the panel. Suspension, reinstatement, deactivation, and AI usage metrics work end to end.

Phase 7 - Complete customer screens and public landing page

Objective: Make the required functionality understandable and usable.

- Customer app: sign-up/login, initial organization and invite flow, members/settings, teams, plan editor, AI review, brain-dump drafting, task ticking, lead approval, team weekly overview, and history.
- Provide loading indicators while AI requests run, useful empty states, clear forbidden pages, and a visible message when AI is unavailable. Remove dead buttons.
- Replace all user-facing Launchbase branding. Keep the internal panel visibly marked as internal.
- Landing page: hero with brand/pitch/Get started, how it works, a real rendered AI plan-review example, benefits for team leads, and a clear call to action leading to sign-up.
- Ensure the site works on phone and desktop, and includes a meaningful HTML title, meta description, Open Graph image, and no placeholder or lorem ipsum copy.

Exit criteria: A new visitor understands the product and can reach sign-up. The main member and lead workflows are usable without dead ends.

Phase 8 - Seed, test, document, package

Objective: Make the reviewer able to explore the product from a fresh local setup.

- Create a manage.py seed command with at least two organizations, several teams, users in different roles, and multiple weeks of plans with a mix of shipped and unshipped goals.
- Provide demo logins for a member, a team lead, an organization owner, and a platform staff user. Do not put real secrets in the repository.
- Run uv run pytest with no Google API key and no outbound Google network access. Run the available frontend checks and exercise the main flows manually.
- Verify forbidden API calls, organization suspension, user deactivation, provider failure, and rate limiting.
- Write SUBMISSION.md with setup instructions, seed command, demo logins, product name/pitch/rationale, actual hours, unfinished items, and what would be fixed with another day.
- Commit all intended files, then create assessment.zip with git archive. Extract it into a fresh folder and follow the instructions before upload.

Exit criteria: A reviewer can start the app, seed it, sign in as each demo role, inspect populated scenarios, and run tests without AI credentials.

7. Verification and acceptance checklist

Use this list as the release gate. A feature is not done just because the page renders. It is done when permitted actions work and prohibited actions are rejected by the backend.

Test group	Scenarios that must pass
Setup and migrations	Fresh database setup, migration upgrade, app startup, repeatable seed command.
Authentication	Sign up with email/password, login, invalid credentials, inactive user, invalid/expired invite handling as implemented.
Tenant isolation	Organization A cannot list, read, modify, or delete organization B data by changing IDs. Repeat for team, plan, task, invitation, and AI usage endpoints.
Roles and teams	Owner/admin/member permissions are enforced. Only the lead assigned to the relevant team can approve or return that team's plans. User can belong to multiple teams.
Plan rules	Monday week key, one plan per member/week, required task types, individual task status, submit/approve/return/done transitions, shipped flag.
AI plan review	Typed output schema validates, findings identify specific tasks, rewrites are useful, suggestions can be applied, provider errors do not crash planning.
AI brain dump	Returns a pre-filled form, never persists automatically, preserves user input on failure.
AI reliability and privacy	Tests run offline with no key. 429/timeouts are handled. Usage is recorded per organization. Secret stays server-side and is not committed.
Internal panel	Customer users are denied access. Staff can view counts, suspend/reinstate organizations, search/deactivate users, view AI usage outcomes.
User experience	Loading and empty states, clear errors/forbidden page, no dead buttons, responsive landing page, brand consistency, title/meta/OG image.
Delivery	SUBMISSION.md is complete. ZIP contains committed source but not node_modules, .venv, api/.env, or uncommitted changes. Extracted archive works from instructions.

8. Risk register and practical responses

Risk	Why it matters	Response
Cross-tenant data leak	Highest-priority assessment area and a serious SaaS security defect.	Centralize authorization helpers, scope every query to membership, add negative tests early, and test IDs from another organization.
Agent-generated code has hidden flaws	A generated route can return data that the UI hides, or mishandle a role.	Review security-sensitive diffs manually. Ask the agent to add a test for every discovered mistake.
Gemini limits or outage	The free tier can rate-limit requests or time out.	Test fallback paths, cap request size, avoid per-keystroke calls, log outcomes, and keep core planning independent from AI.
Overbuilding UI before backend rules	Looks finished but fails cross-role or reviewer tests.	Finish secure API workflows and automated checks first. Use UI to expose working features afterwards.
Generated client gets out of date	Frontend types and backend schemas drift.	Run npm run generate after API changes and use only the typed client conventions in Launchbase.
Time runs out	Scope is broad for a ~12-hour estimate.	Cut optional features and extra polish. Preserve tenancy tests, both AI workflows, staff isolation, seed data, and submission packaging.

9. Optional work, only after the core is complete

The assessment says to choose at most one optional feature. Do not begin these until every required acceptance test passes.

Option	Scope	When to choose it
End-of-week summary	Generate a short note on what shipped, what did not, and why, using the plan and task states.	Best if the current plan/task data model is stable and there is time for one extra typed agent.
Look back before planning	Give an agent access to the member's previous two plans and flag repeated carry-over work.	Best if historical plan retrieval and ownership checks are already reliable.
Team digest	Summarize team risks and identify members who have not submitted their plan.	Best if the team-week overview query is complete and tenant scoping is well tested.

10. Final submission checklist

☐ Product name, one-line pitch, name rationale, and SVG logo are finalized.

☐ Nothing user-facing still says Launchbase. Internal console is visibly marked as internal.

☐ Required signup, organization, team, weekly plan, approval, task tracking, and history paths work.

☐ API tests prove tenant isolation, organization role rules, team-lead approval restrictions, and staff-only internal access.

☐ Both AI workflows use validated typed outputs and have graceful fallback behaviour.

☐ AI usage is recorded by organization. API key is server-side only and excluded from version control.

☐ uv run pytest passes without a Google key or Google network access.

☐ manage.py seed creates the required two-organization demo with teams, roles, and several weeks of plans.

☐ SUBMISSION.md documents how to run the project, seed data, demo logins, brand details, actual hours, and known gaps.

☐ Changes are committed. assessment.zip is created with git archive, extracted into a fresh folder, and checked before upload.

FINAL RULE Do not trade away authorization tests or offline AI tests to add a feature. The assessment explicitly values secure boundaries, useful structured AI behaviour, clear judgement, and an honest record of what remains incomplete.